

构建你的 Superproject

面向复杂系统 Bring-up 的方法学白皮书

从 LinxISA 案例出发，但服务于任何需要组织多模块、多仓库、多团队系统的人

这不是一份关于如何学习 LinxISA 细节的文档。
这是一份关于如何学会构建你自己的 superproject，
如何让复杂系统在多人协作、持续演化和高压验证下仍然保持可收敛、
可复现、可治理的白皮书。

摘要

大多数复杂系统并不是死于单点技术难题，而是死于组织失控：代码分散在多个仓库，边界不清；验证结果散落在聊天记录、日志片段和口头共识中；每次“集成”都像一次没有航图的夜间飞行。系统似乎一直在前进，但没有人真正知道它是否稳定，也没有人能在两周之后解释它为何曾经稳定。

superproject 的价值，就在于把这种混乱重新组织成秩序。它不是一个更大的仓库，也不是一个更长的 README；它更像一座机场塔台：真正飞行的是各架飞机，真正承担空气动力学的是各自机体，但所有飞机的进近顺序、跑道占用、冲突消解、天气通报和起降证据，必须由塔台统一协调。没有塔台，每架飞机都可能在局部上是优秀的，但整个机场一定是危险的。

LinuxISA 提供了一个很强的案例：编译器、模拟器、Linux、RTL、模型、libc、文档、测试矩阵和技能系统并行演进，但最终依靠的不是“多沟通”，而是四个控制平面，即 topology、ownership、validation 与 evidence。这四个平面共同构成了一套可迁移的方法学：任何想要构建自己的 superproject 的团队，都可以借鉴其骨架。

本文的目标，是教会读者学会这种骨架。读完之后，读者应该能够：

- 判断自己是否需要一个真正的 superproject；
- 为自己的系统设计目录与仓库拓扑；
- 建立模块边界、所有权和 repin 纪律；
- 把“测试很多”升级成“严格 gate 管理”；
- 让 agent 与 maintainer 在同一套 machine-readable 规则下工作；
- 把一个复杂系统从“能跑”带到“能长期 bring up”。

目录

摘要	i
第一章 为什么复杂系统需要 Superproject	1
1.1 复杂系统真正难的不是写代码	1
1.2 什么是 Superproject	1
1.3 如果没有它，会发生什么	2
第二章 从 LinuxISA 案例提炼出的四个控制平面	3
2.1 为什么不是一张总表，而是四个平面	3
2.2 Topology：先决定道路，不要先讨论交通	3
2.3 Ownership：没有明确负责者，系统就没有真正的记忆	3
2.4 Validation：把“我们觉得差不多了”升级为严格 gate	4
2.5 Evidence：如果没有证据包，绿色就只是情绪	4
第三章 从案例到通用方法：如何构建你自己的 Superproject	5
3.1 第一步：画出模块地图，而不是先写总脚本	5
3.2 第二步：为每个模块定义 closure surface	5
3.3 第三步：把所有权写成机器能懂的格式	6
3.4 第四步：建立双层验证，而不是把所有检查塞进一个脚本	6
3.4.1 静态层	6
3.4.2 运行时层	6
3.5 第五步：设计 Repin Discipline	7
3.6 第六步：为系统设计证据包	7
第四章 Strict Gate、Strict Check 与 Test-Driven Bring-up 的通用原则	8
4.1 测试很多，不等于严格	8
4.2 Strict Check 是给流程本身加护栏	8
4.3 Test-Driven Bring-up 的关键不是先写测试，而是先定义失败面	8
第五章 如何真正学会这套方法学	10
5.1 先学会看边界，再学会看工具	10
5.2 把系统想象成一座城市	10
5.3 用三层练习法掌握它	10
5.3.1 第一层：读图	10
5.3.2 第二层：跟跑	11

5.3.3 第三层：造骨架	11
第六章 给读者的落地蓝图：如何为你的项目搭骨架	12
6.1 最小可行 Superproject 套件	12
6.2 建议的目录雏形	12
6.3 建议的最小字段	13
6.4 两条最重要的落地建议	13
6.4.1 不要一开始追求“大而全”	13
6.4.2 不要让流程只存在于“资深工程师知道”的层面	14
第七章 新 Agent 的通用 SOP 模板	15
7.1 使命定义	15
7.2 进入系统时的第一组动作	15
7.3 新 Agent 的五条戒律	15
7.4 推荐的标准闭环	15
第八章 Maintainer 的通用 Repin Checklist 模板	17
8.1 Repin 前先问自己	17
8.2 Maintainer 的八项合并前核对	17
8.3 一句最重要的提醒	17
第九章 常见反模式	18
9.1 路径漂移	18
9.2 状态页先行	18
9.3 把多模块一起顺手升级	18
9.4 把资深工程师当作方法学本身	18
第十章 结论：如何判断你的 Superproject 已经成熟	19
附录：可直接借用的最小命令集合	20

第 1 章 为什么复杂系统需要 Superproject

1.1 复杂系统真正难的不是写代码

单个模块的复杂性，往往还能够由模块自己的代码结构、测试框架和维护者经验来承受。但一旦系统演化到多仓库、多语言、多执行面、多团队协作的阶段，难点会发生转移。真正的挑战不再是“这一段代码如何实现”，而变成：

- 哪个模块应该承担哪个语义；
- 当合同变化时，谁必须同步；
- 哪些测试能代表局部正确，哪些 gate 才代表系统正确；
- 哪次失败是叶子模块的责任，哪次失败是集成层的责任；
- 今天的绿色结果，明天是否还能被复现。

如果说单模块工程的重点是“造船”，那么 superproject 面对的是“建港”。造船需要处理材料、结构和动力；建港则必须同时处理航道、泊位、调度、验收、维修和记录。没有港口治理，再好的船也会互相阻塞。

1.2 什么是 Superproject

superproject 不是“所有代码都在一个仓库里”的同义词。真正的 superproject 至少有四个特征：

1. 它协调多个真实独立的实现单元；
2. 它维护这些单元之间的合同、版本关系与验证关系；
3. 它拥有比单个模块更高一级的 gate 和证据模型；
4. 它能够回答“整个系统在什么意义上是绿的”。

在这个定义下，superproject 更像一部宪法，而不是一部法典全集。它不亲自规定每个模块内部如何写每一行代码，但它规定边界、职责、冲突解决方式和通过标准。

1.3 如果没有它，会发生什么

当系统没有 superproject 方法学时，团队通常会落入三种幻觉：

- **局部绿色幻觉**：每个模块都能给出自己的“通过”截图，但没有人能证明整个系统一起通过；
- **近期可用幻觉**：今天的工作树可以运行，但一周后无法复现，也不知道究竟是谁改变了什么；
- **责任模糊幻觉**：一旦出现回归，所有人都觉得自己“可能没问题”，最后没人真正负责系统收敛。

这三种幻觉，正是 superproject 要消除的对象。

第 2 章 从 LinuxISA 案例提炼出的四个控制平面

2.1 为什么不是一张总表，而是四个平面

很多项目会试图用一张总表管理一切：模块列表、测试结果、负责人、进展备注、风险提示都混在一起。这样的表在最初看起来很方便，但当系统规模增长后，它会变成一块巨大的湿泥地：什么信息都在上面，却没有任何一种信息能被严格验证。

LinuxISA 的经验表明，更有效的做法是把治理拆成四个平面。它们像建筑中的四个不同层次：地基、承重结构、安检系统和监控系统。你不能指望用监控摄像头去替代承重墙。

2.2 Topology：先决定道路，不要先讨论交通

Topology 平面关注路径、目录、仓库链接和 canonical location。它回答的是：哪些地方允许存在，哪些地方禁止存在，新的工作应落到哪里。

这是方法学里最容易被轻视的一层，因为它看起来不像“高技术”。但事实上，topology 是复杂系统的地形学。没有明确地形，所有后续治理都会漂浮。

LinuxISA 的做法很有代表性：

- 用根目录 `.gitmodules` 统一维护 submodule 拓扑；
- 用 `AGENTS.md` 和 `docs/project/navigation.md` 声明允许路径与禁止路径；
- 用 `tools/ci/check_repo_layout.sh` 把目录约束变成 CI 能执行的规则。

可迁移原则是：先定义地形，再允许建设。如果一个团队在 topology 上是放任的，那么再严格的 test discipline 也只能在流沙上行走。

2.3 Ownership：没有明确负责人，系统就没有真正的记忆

Ownership 平面不是“谁最懂某个模块”，而是“当某种失败发生时，系统知道应该找谁”。它要求责任边界能被机器读取，而不是只存在于人脑中。

在 LinuxISA 里，这一层通过 `manifest.yaml`、`checklists` 和 `waiver ledger` 来实现。它的启发是：所有权必须结构化，才能规模化。

如果你的系统没有 ownership 平面，团队就像一支没有站位表的足球队：每个人都在跑，但没有人知道哪个球是自己必须接的。

2.4 Validation：把“我们觉得差不多了”升级为严格 gate

Validation 平面是 superproject 的安检系统。它不问“代码多不多”，也不问“大家辛不辛苦”，只问一件事：系统是否满足进入下一阶段的客观条件。

LinuxISA 的验证层不是一个单一脚本，而是一组彼此配合的脚本：

- 结构性静态检查；
- AVS 合同检查；
- PR-tier 与 nightly-tier strict closure；
- dual-lane runtime convergence；
- gate consistency 与 freshness 审计。

这说明一个重要原则：**复杂系统的验证不是一把尺子，而是一套闸门**。就像水库闸门一样，它们不是为了“限制你前进”，而是为了保证系统在更高水位下不被冲垮。

2.5 Evidence：如果没有证据包，绿色就只是情绪

Evidence 平面让系统具备可追溯性。它要求每次重要 run 都留下足够完整的证据：命令、lane、run-id、SHA manifest、log 路径、summary JSON，以及与状态页一致的 canonical report。

LinuxISA 的经验在这里尤其值得借鉴：它明确把 markdown 状态页定义为“派生视图”，而不是 source of truth。这一选择很重要，因为它切断了“手动修 status page”这种危险捷径。

你可以把 evidence 看成系统的黑匣子。黑匣子不是为了日常美观，而是为了在出现失败时，系统仍然有记忆。

第 3 章 从案例到通用方法：如何构建你自己的 Superproject

3.1 第一步：画出模块地图，而不是先写总脚本

很多团队上来就想写一个 `run_all.sh`，仿佛只要能一键执行，一切就会自然有序。实际上，一键执行只是最后的皮肤；真正的秩序来自于你先知道系统有哪些模块、边界在哪里、哪些关系是硬约束、哪些关系只是调用关系。

建议用一张最小模块地图回答下面问题：

1. 这个系统有哪些一等模块；
2. 哪些模块是实现模块，哪些模块是协调模块；
3. 哪些模块的变化会引发跨模块验证；
4. 哪些目录或仓库是 canonical location；
5. 哪些历史路径必须禁止复活。

3.2 第二步：为每个模块定义 closure surface

一个健康的 superproject 不会要求每个模块做“所有测试”，而是要求每个模块提供可解释的 closure surface。所谓 closure surface，就是“什么样的证据足以证明这个模块在当前阶段是健康的”。例如：

- 编译器模块可以是 compile suites、coverage 和工具链闭包；
- 模拟器模块可以是 runtime suites、strict system 和 opcode sync；
- kernel 模块可以是 build closure、smoke boot 和 full boot；
- 文档/架构模块可以是 contract lint 和 matrix closure。

如果模块没有 closure surface，它就无法被稳定地集成；它只能以“作者自己觉得没问题”的形式进入系统。这是大多数 superproject 失控的起点。

3.3 第三步：把所有权写成机器能懂的格式

很多团队会说“这个模块归 Alice 管，那个模块归 Bob 管”。这不够。真正需要的是：

- 哪个 gate 属于谁；
- 哪个 checklist 代表其责任完成；
- 哪些 agent 允许跨模块；
- 哪类失败可以被 waiver，谁批准，何时过期。

一旦写成机器能读取的结构，维护者和 agent 就能在同一套语义下工作。否则，superproject 规模一增长，就只能靠资深工程师脑中的“暗知识”维持。

3.4 第四步：建立双层验证，而不是把所有检查塞进一个脚本

最可迁移的设计之一，是把验证分成静态层和运行时层。

3.4.1 静态层

静态层关注治理结构本身是否成立。例如：

- manifest 是否合法；
- gate 是否都有 owner；
- checklist ID 是否冲突；
- waivers 是否带过期时间；
- 模块 scope 是否超界。

3.4.2 运行时层

运行时层关注某个具体 run 是否满足通过条件。例如：

- 本次 run 的 required row 是否全绿；
- lane 和 run-id 是否完整；
- summary JSON 是否存在；
- 证据是否新鲜且路径正确。

把这两层混在一起，会导致一个典型问题：测试失败时，你根本不知道失败的是“系统行为”还是“治理结构”。分层之后，诊断会清晰很多。

3.5 第五步：设计 Repin Discipline

repin 是 superproject 最危险、也最能暴露成熟度的动作。它看起来只是移动几个 SHA，但实际上等于宣布“这些独立模块现在可以共同作为一个系统被信任”。

因此，repin 需要纪律：

1. 叶子模块先修复并先证明；
2. 再在 superproject 中更新 submodule SHA；
3. 再执行严格 cross-repo closure；
4. 最后才允许合并。

简而言之：repin 不是同步动作，而是发布动作。

3.6 第六步：为系统设计证据包

想让系统具备长期可维护性，你必须让每一次重要结果都能被重新讲述。方法不是要求大家写更长的总结，而是要求每次 run 都生成证据包。

一个最小证据包应包括：

- run-id；
- lane；
- 精确命令；
- SHA manifest；
- canonical JSON 中的结果行；
- 对应的日志和 summary artifact。

一旦有了这种结构，沟通成本会下降很多，因为很多争论都可以回到证据而不是回到记忆。

第 4 章 Strict Gate、Strict Check 与 Test-Driven Bring-up 的通用原则

4.1 测试很多，不等于严格

很多项目测试非常多，但仍然不严格。原因在于“测试集合”与“通过标准”之间缺了一层治理。strict gate 的本质不是增加测试数量，而是把通过条件形式化。

这意味着：

- 哪些 gate 是 required；
- 哪些 gate 在哪个 phase 才算 blocker；
- waiver 何时合法；
- 哪些结果算 stale；
- 什么样的 lane policy 才允许宣称系统收敛。

4.2 Strict Check 是给流程本身加护栏

strict check 的核心价值，是把“流程是否被正确执行”也变成验证对象。这样一来，系统不会因为“大家都以为别人会补证据”而出现治理漏洞。

从方法学角度看，strict check 有三个作用：

1. 它让流程可审计；
2. 它让团队不依赖少数资深人员的记忆；
3. 它让 agent 能在清晰边界内自主执行。

4.3 Test-Driven Bring-up 的关键不是先写测试，而是先定义失败面

很多人把 test-driven 理解成“先写测试再写实现”。在 bring-up 场景里，更准确的表达是：先定义失败面，再组织修复闭环。

一个成熟的 bring-up loop 应该是：

1. 从失败 contract、audit、gate 或 regression 开始；
2. 用最小命令复现；

3. 在正确的 owning module 内修复；
4. 用该模块的 closure surface 验证修复；
5. 再执行 superproject 集成验证。

如果没有这个 loop，项目就会不断累积“看起来应该没问题”的改动，最终在集成时一起爆炸。

第 5 章 如何真正学会这套方法学

5.1 先学会看边界，再学会看工具

很多读者在学习 superproject 方法学时，第一反应是去记命令和脚本名字。这当然有用，但那只是表层。真正应该先学会的是边界感。

你需要训练自己反复问四个问题：

1. 这个问题属于哪个模块；
2. 这个模块的 closure surface 是什么；
3. 这个失败会不会影响 cross-repo 集成；
4. 这次结果的权威证据在哪里。

一旦这四个问题成为你的直觉，工具就不再是负担，而会自然落位。

5.2 把系统想象成一座城市

如果你更习惯通过比喻学习，可以把 superproject 想象成一座城市：

- 模块是城市里的功能区；
- topology 是道路和规划红线；
- ownership 是各区的行政与维护责任；
- validation 是安检、消防和准入制度；
- evidence 是城市的监控、档案与事故记录；
- repin 则像修改城市公共基础设施的生效版本。

在这样的城市里，真正危险的不是一栋楼有瑕疵，而是消防、道路、产权和档案同时失效。superproject 的方法学，就是防止这类系统性塌方。

5.3 用三层练习法掌握它

5.3.1 第一层：读图

先不急着改系统。先能读懂：

- 模块地图；
- gate ownership；
- lane 设计；
- artifact 布局；
- waiver 规则。

5.3.2 第二层：跟跑

选择一条失败的 gate，跟着现有流程走完整一遍：

1. 复现；
2. 定位 owning module；
3. 修复；
4. 重跑；
5. 留证据；
6. 判断是否 repin。

5.3.3 第三层：造骨架

等你能稳定完成第二层，再尝试为自己的项目亲手建立这些文件：

- navigation contract；
- module manifest；
- checklists；
- waiver ledger；
- canonical report schema；
- PR/nightly gate entrypoint；
- maintainer repin checklist；
- agent SOP。

真正学会方法学，不是把别人的脚本背下来，而是能为自己的系统造出同等强度的骨架。

第 6 章 给读者的落地蓝图：如何为你的项目搭骨架

6.1 最小可行 Superproject 套件

如果你要为自己的项目从零开始搭建，建议最少准备八类文件或入口：

1. **导航合同**：声明允许路径、禁止路径、canonical location；
2. **模块清单**：列出模块、职责、边界；
3. **ownership manifest**：列出 gate 与 owner 的映射；
4. **checklists**：给每个域定义可审计的完成标准；
5. **waiver ledger**：记录何时允许带着风险前进；
6. **strict entrypoint**：PR-tier 和 nightly-tier 入口脚本；
7. **canonical report**：机器可读的 gate 报告；
8. **agent / maintainer runbook**：给执行者清晰 SOP。

6.2 建议的目录雏形

下面是一种可迁移的最小目录骨架：

```
superproject/  
  modules/  
  docs/project/  
    navigation.md  
    repository-flow.md  
    ownership-manifest.yaml  
    waivers.yaml  
    new-agent-sop.md  
    maintainer-repin-checklist.md  
  docs/gates/  
    latest.json  
    logs/  
  tools/ci/  
    check_repo_layout.sh
```



```
tools/bringup/  
  check_static_policy.py  
  check_runtime_closure.py  
  check_gate_consistency.py  
tools/regression/  
  strict_cross_repo.sh
```

6.3 建议的最小字段

ownership manifest 至少应包含：

- 模块名；
- owner；
- 允许修改的路径；
- 对应的 checklist；
- gate key 与其归属；
- 哪些角色允许跨模块。

waiver ledger 至少应包含：

- gate key；
- owner；
- issue 或原因；
- phase；
- expires_utc；
- 是否仍 active。

6.4 两条最重要的落地建议

6.4.1 不要一开始追求“大而全”

真正有效的 superproject 往往从很少的硬约束开始，例如：

- 禁止路径漂移；
- 每个 gate 必须有 owner；
- canonical JSON 是最高真相；
- repin 必须在 leaf proof 之后。

先把这四条做硬，再逐步增加复杂度。

6.4.2 不要让流程只存在于“资深工程师知道”的层面

如果一个系统的关键治理规则无法被新成员或 agent 从仓库中直接读到，这个系统迟早会在规模增长后失去稳定性。方法学真正成熟的标志，不是“老板觉得放心”，而是“新人能依靠文档和脚本走到正确位置”。

第 7 章 新 Agent 的通用 SOP 模板

7.1 使命定义

一个新 agent 的第一职责不是“尽快做出修改”，而是先对系统秩序负责。具体来说，它应做到：

1. 保持在 canonical workspace 与声明过的 module scope 内；
2. 找到最小失败面；
3. 优先修 leaf module；
4. 再考虑 repin 与 strict closure；
5. 留下可复核证据。

7.2 进入系统时的第一组动作

```
git submodule sync --recursive
git submodule update --init --recursive
bash tools/ci/check_repo_layout.sh
```

如果 layout 不通过，后续运行结果一律先打折扣。

7.3 新 Agent 的五条戒律

- 不在未声明模块中越界修改；
- 不在 superproject 层掩盖叶子模块缺陷；
- 不手改派生状态页；
- 不把无 waiver 的红 gate 当作“暂时可接受”；
- 不在证据缺失时宣称闭环完成。

7.4 推荐的标准闭环

1. 读取导航合同、manifest 与 checklist；
2. 用最小命令复现失败；

3. 路由到 owning module;
4. 修复并重跑 leaf gate;
5. 如需要, repin;
6. 运行 strict cross-repo closure;
7. 检查 artifact 与 canonical report。

第 8 章 Maintainer 的通用 Repin Checklist 模板

8.1 Repin 前先问自己

在移动任何 SHA 之前，maintainer 应先问：

1. 这次 repin 的业务或技术原因是什么；
2. 对应 leaf proof 是什么；
3. 哪些 cross-repo gate 可能受影响；
4. 这是一笔“明确的发布动作”，还是只是“顺手同步”。

如果答案偏向后者，应该停下来压缩范围。

8.2 Maintainer 的八项合并前核对

1. intended modules 是本批次唯一发生 SHA 变化的模块；
2. 每个模块都有 leaf proof；
3. PR-tier 或 nightly-tier strict closure 已执行；
4. 需要时完成 dual-lane 验证；
5. canonical report 是新鲜的；
6. 派生状态页与 canonical report 一致；
7. waiver 已审查；
8. commit message 能解释 repin 原因与范围。

8.3 一句最重要的提醒

maintainer 不应把 repin 当作“仓库同步”。repin 在方法学上更接近一次小型发布：它向系统宣布一组版本关系开始生效，因此必须带着证据和纪律进入主线。

第 9 章 常见反模式

9.1 路径漂移

项目越做越久，大家越容易为了“先做成”而在仓库边缘长出许多近似目录。短期看很灵活，长期看等于让 topology 平面失效。

9.2 状态页先行

一些团队会先改进度页、再补证据，仿佛文字能替代事实。这种做法的危险之处在于，它会把整个系统训练成“先讲故事，再找证据”。

9.3 把多模块一起顺手升级

这是最典型的 repin 失控模式。它让任何失败都变得难以归因，也让任何成功都失去可复现性。

9.4 把资深工程师当作方法学本身

如果一位关键工程师请假，系统就几乎无法完成闭环，那么你拥有的不是成熟的方法学，而只是一个高负载的人类路由器。

第 10 章 结论：如何判断你的 Superproject 已经成熟

你可以用下面五个问题来审视自己的系统：

1. 新成员是否能在不依赖口头传承的前提下找到正确路径；
2. 每个关键 gate 是否都有明确 owner；
3. 每次重要 run 是否都有完整证据包；
4. repin 是否始终建立在 leaf proof 之上；
5. 系统是否能说清“整体绿色”的定义。

如果这五个问题中大部分答案仍然含糊，那么说明你可能拥有多个仓库，但还没有真正拥有一个 superproject。

而一旦这些问题都能被清楚回答，你就不只是拥有了一套 bring-up 流程，而是拥有了一套能在复杂度增长中继续生存的方法学。这正是 superproject 的意义：它不是为了让系统看起来更宏大，而是为了让系统在规模变大之后仍然不失控。

附录：可直接借用的最小命令集合

1. 初始化与 layout 检查

```
git submodule sync --recursive
git submodule update --init --recursive
bash tools/ci/check_repo_layout.sh
```

2. 静态策略检查

```
python3 tools/bringup/check_multi_agent_gates.py --strict-always --mode
static --manifest docs/bringup/agent_runs/manifest.yaml --waivers
docs/bringup/agent_runs/waivers.yaml --checklists-root
docs/bringup/agent_runs/checklists
```

3. 合同检查

```
python3 tools/bringup/check_avs_contract.py --matrix
avs/linux_avs_v1_test_matrix.yaml
python3 tools/bringup/check_avs_profile_closure.py --matrix
avs/linux_avs_v1_test_matrix.yaml --status
avs/linux_avs_v1_test_matrix_status.json --tier pr
```

4. PR-tier strict closure

```
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/regression/strict_cross_repo.sh
```

5. Dual-lane convergence

```
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/bringup/run_runtime_convergence.sh --lane pin --run-id <run-id-pin>
LINUX_GATE_TIER=pr RUN_EXTENDED_CROSS_GATES=1 bash
tools/bringup/run_runtime_convergence.sh --lane external --run-id
<run-id-external>
```